HackenProof

Expert crowdsourced security provider

REPORT

Audit

for **CurrenC (CRNC) Token Contract**

PREPARED BY

HackenProof

REPORT CREATED

24.04.2026

AUDIT DURATION

23.04.26-24.04.26

AUDIT TYPE

Solidity

HackenProof

TABLE OF CONTENTS

Introduction 3

Audit summary 4

Findings list 4

About project 16

HackenProof

INTRODUCTION

HackenProof Audit is your solution to maximize cybersecurity through the involvement of the industry's most skilled and experienced community security researchers.

Name	CurrenC
Website	https://currenc.co
Type	Audit
Date	23.04.26-24.04.26
Scope	https://github.com/crnc-token/CRNC
Approved by	HackenProof team
Audited by	 @tchkvsky  @0xBugSlayer

HackenProof

AUDIT SUMMARY

0	0	3
Total Findings	Resolved	Informative

• Critical	0
• High	0
• Medium	0
• Low	0
• None	3

FINDINGS LIST

- NONE

CURREN-3 CRNC Review Report	Informative
CURREN-2 The token total supply is over the owner's control	Informative
CURREN-1 The token lacks burning functionality	Informative

HackenProof

VULNERABILITY DETAILS

CURREN-3 CRNC Review Report

• NONE

Summary: CurrenC is a fixed-supply ERC20 built on an unmodified OpenZeppelin v5 implementation. The contract's entire logic is a constructor that mints 21,000,000 CRNC to the deployer. Review of the runtime surfaced no Low, Medium, High, or Critical issues.

Five Informational items are recorded, all actionable at the repository, documentation, or test level. None affect the deployed contract at 0x47d13fb0803409c7fe169a210d4e906165f1a432.

I-01: Redundant TOTAL_SUPPLY() public getter
I-02: Constructor uses msg.sender as implicit recipient Informational
I-03: Floating source pragma
I-04: Test suite assurance gaps
I-05: Documentation inconsistencies on tests and off-chain claims

Scope:
- contracts/CurrenC.sol
- test/CurrenC.t.sol
- foundry.toml
- README.md, SECURITY.md, DEPLOYMENT.md, AUDIT_NOTES.md, WHITEPAPER.md
- audit/*

Status: Informative

Severity: • NONE

Vulnerability details

I-01. **Redundant TOTAL_SUPPLY() public getter**

Category: Gas/ABI surface

Location:
- contracts/CurrenC.sol:19
- audit/foundry-abi.txt:21

Description: TOTAL_SUPPLY is declared public constant, so the compiler synthesises a TOTAL_SUPPLY() external getter at selector 0x902d55a5. The OZ ERC20 that CurrenC inherits already provides totalSupply() at selector 0x18160ddd, and both return the same value.

```
uint256 public constant TOTAL_SUPPLY = 21_000_000 * 10 ** 18;
```

Impact: The extra function adds to the dispatch table and the deployed bytecode. It also clutters the ABI: consumers iterating the interface see two selectors returning the same value and cannot tell which one the contract considers authoritative.

Bytecode cost before and after flipping visibility to internal (forge build --sizes):

Variant	Runtime (B)	Initcode (B)
public constant TOTAL_SUPPLY	1,792	2,675
internal constant TOTAL_SUPPLY	1,763	2,646
Delta	**-29**	**-29**

The existing five tests continue to pass after the change; no test calls token.TOTAL_SUPPLY() via the public getter.

Recommendation:

If the constant only exists to name the literal, declare it internal (or private). If the named getter is genuinely intended as part of the public surface, keep public and add an inline comment. The current source does not make the choice explicit.

I-02. **Constructor uses msg.sender as implicit recipient**

Category: Deployment/design

Location:

- contracts/CurrenC.sol:21-22
- DEPLOYMENT.md:20
- AUDIT_NOTES.md:64
- test/CurrenC.t.sol:12-16

Description: The constructor takes no arguments and mints the full supply to msg.sender:

```
constructor() ERC20("CurrenC", "CRNC") {
    _mint(msg.sender, TOTAL_SUPPLY);
}
```

There is no initialHolder parameter, no explicit check on the caller, and no comment marking msg.sender as a deliberate decision rather than a default. OZ's _mint guards only against to == address(0), but it does not look at msg.sender. The mint is one-shot and nothing in the contract can reallocate the initial supply later.

Impact: The msg.sender choice is invisible to any deployment path that routes through an intermediate contract. Three realistic examples:

- Deployment through a CREATE2 factory -> The factory contract receives the full 21M balance.
- Deployment from a Safe via an authorised module -> The module address receives the balance, not the Safe.
- Deployment through an ERC-2771 forwarder -> The forwarder receives the balance.

None of these recipients are the zero address, so OZ's zero-receiver guard does not intervene.

Recommendation:

Take the recipient as an explicit parameter and validate it:

```
constructor(address initialHolder) ERC20("CurrenC", "CRNC") {
    require(initialHolder != address(0), "zero holder");
    _mint(initialHolder, TOTAL_SUPPLY);
}
```

If msg.sender is the deliberate design choice, state it inline so the reasoning survives copy-paste into another project:

```
// intentional: deploy must be direct-from-EOA or Safe; not via factory/forwarder
_mint(msg.sender, TOTAL_SUPPLY);
```

I-03. **Floating source pragma**

Category: Code quality/reproducibility

Location:

- contracts/CurrenC.sol:2 (pragma solidity ^0.8.28;)
- foundry.toml:4 (solc_version = "0.8.28")

Description: foundry.toml pins the project compiler to 0.8.28, but the source pragma (^0.8.28) accepts any later 0.8.x release. A downstream builder using a different patch version will compile this source without complaint and produce bytecode that does not match the deployed contract.

Impact: Bytecode reproducibility from a clean checkout is not guaranteed. Any future Etherscan re-verification can silently succeed against a compiler version other than the one originally used.

Recommendation:

Pin the source pragma: pragma solidity 0.8.28;

I-04. **Test suite assurance gaps**

Category: Assurance quality

Location:

- test/CurrenC.t.sol

Description: The test suite exercises the happy paths (supply, transfer, and transferFrom), but does not exercise the properties the documentation says it verifies. Four specific gaps:

1. testNoMintFunctionExists() is a placeholder. Its body is assertTrue(true);, it passes unconditionally, and the comment explaining why is wrong: adding a mint() function to CurrenC would compile without error.
2. The documentation claims the suite verifies absence of mint, burn, and administrative functions, but no test issues a low-level call against any of the relevant selectors: mint(address,uint256), burn(uint256), owner(), pause(), transferOwnership(address).

3. The two happy-path tests ignore the return value of transfer and transferFrom. Forge's erc20-unchecked-transfer lint flags both call sites (test/CurrenC.t.sol:35 and :49); nothing in the tests breaks if either call returns false.
4. testApproveAndTransferFromWorks never checks that the allowance is consumed. It verifies Alice's balance after the call, but not that allowance(deployer, alice) dropped to zero.

Impact: The suite does not exercise the properties the repository claims are covered. A regression that reintroduced an admin function or quietly broke allowance accounting would still pass.

Recommendation:

- Replace testNoMintFunctionExists with a negative-selector probe that staticcalls each of mint(address,uint256), burn(uint256), burnFrom(address,uint256), owner(), pause(), unpause(), transferOwnership(address) and asserts the low-level call returns ok == false.
- Check the return values:
- assertTrue(token.transfer(alice, amount));
- assertTrue(token.transferFrom(deployer, alice, amount));
- After transferFrom, assert allowance(deployer, alice) == 0.

I-05. **Documentation inconsistencies on tests and off-chain claims**

Category: Documentation/framing

Location:

- README.md:63, 81-86
- WHITEPAPER.md:54-56
- SECURITY.md:69-81, 122-127
- DEPLOYMENT.md:43-46
- AUDIT_NOTES.md:177-188

Description: Two factual inconsistencies run through the documentation:

1. Off-chain reliance. README.md lists "No reliance on off-chain enforcement" as a design principle and WHITEPAPER.md states "All properties of CRNC are enforced by code and irreversible on-chain actions". SECURITY.md and DEPLOYMENT.md describe the opposite — liquidity provisioning and the LP burn are external, manual steps the token contract does not and cannot enforce.
2. Test coverage. README.md, SECURITY.md, and AUDIT_NOTES.md state that the Foundry tests validate "absence of mint and burn functionality" and "absence of administrative controls". They do not (see I-04).

Impact: A reader relying on the public-facing documents (README, Whitepaper) infers a stronger trust model than the system provides. The gap is sharpest around liquidity permanence which is presented as an on-chain property but is dependent on the deployer correctly executing pool creation and LP burn.

Recommendation:

Narrow the "on-chain enforced" language in README.md and WHITEPAPER.md so it only covers properties the token actually enforces (supply fixity, no admin). Frame liquidity permanence as a deployment claim verifiable on-chain, not an on-chain invariant.
Either pare the coverage claims back to what the current suite actually tests, or expand the suite per I-04.

Nits

- audit/README.md does not record the commands used to produce the seven committed artifacts. A short Makefile or justfile recipe per artifact removes the ambiguity.
- Slither's output can be cleaned up with a slither.config.json containing filter_paths = "node_modules|lib"; every current finding in audit/slither-report.txt concerns OZ library pragma ranges rather than CRNC code.
- lib/forge-std is vendored as loose files rather than a pinned submodule. Record the release tag or convert to a submodule so the checked-in version is reproducible.

Validation steps

Per-finding reproduction and verification steps for the CRNC review. Each block mirrors the numbering in the main report.

I-01. Redundant TOTAL_SUPPLY() public getter

1. The duplicate selector appears in audit/foundry-abi.txt:21: TOTAL_SUPPLY() at 0x902d55a5 alongside totalSupply() at 0x18160ddd.
2. Both are callable on the deployed contract. cast call 0x47d13fb0803409c7fe169a210d4e906165f1a432 "TOTAL_SUPPLY()(uint256)" and "totalSupply()(uint256)" each return 21000000000000000000000000.
3. The bytecode cost is reproducible locally. forge build --sizes reports 1,792 B runtime / 2,675 B initcode on the current source; changing the declaration at contracts/CurrenC.sol:19 to internal constant and rebuilding reports 1,763 B / 2,646 B.

I-02. Constructor uses msg.sender as implicit recipient

1. The constructor at contracts/CurrenC.sol:21-22 accepts no recipient parameter and calls _mint(msg.sender, TOTAL_SUPPLY).

2. A Foundry test reproduces the misroute under indirect deployment:

```
contract Factory {
    function deploy() external returns (CurrenC t) { t = new CurrenC(); }
}

function testFactoryReceivesSupply() public {
    Factory f = new Factory();
    CurrenC t = f.deploy();
    assertEq(t.balanceOf(address(f)), 21_000_000e18);
    assertEq(t.balanceOf(address(this)), 0);
}
```

Both assertions pass: 100% of supply is held by the factory, not by the caller that initiated deployment.

I-03. Floating source pragma

1. contracts/CurrenC.sol:2 declares pragma solidity ^0.8.28;.
2. foundry.toml:4 sets solc_version = "0.8.28", which pins only the project-local build.
3. Setting foundry.toml to any later 0.8.x (e.g. 0.8.30) and running forge build produces a successful compilation whose forge inspect CurrenC bytecode output differs from the 0.8.28 build.

I-04. Test suite assurance gaps

1. test/CurrenC.t.sol:56-61 defines testNoMintFunctionExists() with assertTrue(true); as its entire body.
2. forge build --sizes emits erc20-unchecked-transfer warnings against test/CurrenC.t.sol:35 and test/CurrenC.t.sol:49.
3. The absence of revert-path, allowance-consumption, and forbidden-selector assertions is visible in the source:

```
grep -n "allowance" test/CurrenC.t.sol
grep -En "expectRevert|expectEmit" test/CurrenC.t.sol
grep -Ein "mint\(|burn\(|owner\(|pause\(|transferOwnership" test/CurrenC.t.sol
```

No match is returned.

The coverage claims in README.md:81-86, SECURITY.md:122-127, and AUDIT_NOTES.md:177-188 are consequently unsupported by the suite in its current form.

I-05. Documentation inconsistencies on tests and off-chain claims

1. README.md:63 — "No reliance on off-chain enforcement".
2. WHITEPAPER.md:54-56 — "All properties of CRNC are enforced by code and irreversible on-chain actions".
3. SECURITY.md:69-81 — the deployer "must correctly … perform the intended initial liquidity provisioning".
4. DEPLOYMENT.md:43-46 — "This step is performed manually by the deployer and is not enforced by the token contract."
5. Statements (1)–(2) contradict (3)–(4): off-chain deployer action is required, but two public-facing documents assert the opposite.
6. test/CurrenC.t.sol contains no tests for the absence of mint, burn, or administrative controls, contradicting the coverage claims in README.md:81-86, SECURITY.md:122-127, and AUDIT_NOTES.md:177-188.

Comments history

CurrenC team

All findings are acknowledged. CurrenC appreciates the detailed review and classification of these items as informational.

The contract is intentionally minimal, with its behavior defined entirely at deployment. No administrative, upgradeable, minting, or burning mechanisms exist by design. The objective is to minimize surface area and eliminate discretionary control at the contract level.

Regarding specific findings:

I-01 (TOTAL_SUPPLY getter):
The duplicate getter does not impact runtime behavior or correctness of the deployed contract. It is acknowledged as an ABI and bytecode surface consideration. The deployed contract remains unchanged.

I-02 (msg.sender as initial recipient):
The use of msg.sender is intentional and reflects a direct deployment model. The contract is designed to be deployed from a controlled address, followed by immediate distribution via liquidity provisioning. This behavior is deliberate and fundamental to the deployment sequence, and does not represent an ongoing control condition.

I-03 (pragma version):
The floating pragma is acknowledged as a reproducibility consideration. The deployed contract is fixed and verifiable on-chain; compiler version alignment applies only to external reproduction of the source.

I-04 (test coverage):
The current test suite focuses on core ERC-20 functionality. The gap between documented claims and test coverage is acknowledged as a repository-level consideration. The deployed contract behavior is unaffected.

I-05 (documentation consistency):
This is acknowledged as the most important clarification.
The contract enforces supply immutability and absence of administrative control on-chain. Liquidity provisioning and LP token burning are external deployment actions that are verifiable on-chain but not enforced by the contract itself.

CurrenC's public documentation distinguishes between:
On-chain enforced properties (fixed supply, no admin functions)
Deployment actions verifiable on-chain (liquidity provisioning and LP burn)

No changes are required to the deployed contract at 0x47d13fb0803409c7fe169a210d4e906165f1a432, as none of the findings impact runtime behavior or introduce security risk.

These findings are valuable in improving clarity, reproducibility context, and documentation accuracy, without affecting the integrity of the deployed system.

VULNERABILITY DETAILS

CURREN-2 The token total supply is over the owner's control • NONE

Scope: https://github.com/crnc-token/CRNC

Status: Informative

Severity: • NONE

Vulnerability details

Although this is an intended behaviour, since the owner will be the depositor of the the tokens to UniV2, he has a full control over the whole total supply of tokens, which may stop people from using the token (since it is centralised).

A way to mitigate this is to add some functionality to the contract. For example mint the tokens and deposit them from the contract. This way nobody will really have full control over the tokens, making it more decentralized and trustworthy.

Validation steps

1. The whole total supply is minted to the owner
2. The owner will be the liquidity provider for UniV2, meaning he has full control over the whole supply.

Comments history

CurrenC team

The initial allocation of the full token supply to the deployer at deployment is intentional and fundamental to the contract design.

The supply is fixed at deployment, with no minting, burning, or administrative functions thereafter.

While the deployer initially received the supply, distribution was completed immediately through liquidity provisioning:
90% of supply was deposited into a Uniswap v2 pool
LP tokens were sent to a burn address, permanently locking liquidity

This state existed only during the initial deployment sequence and was not an ongoing condition following liquidity lock.

As a result, no further control over supply or distribution exists at the contract level.

This approach avoids introducing additional contract logic, reducing attack surface and improving auditability.

Accordingly, this does not represent an ongoing centralization risk, but rather a temporary deployment step followed by irreversible decentralization via locked liquidity.

VULNERABILITY DETAILS

CURREN-1 The token lacks burning functionality

• NONE

Scope: https://github.com/crnc-token/CRNC

Status: Informative

Severity: • NONE

Vulnerability details

Since 90% of the token supply will be used as UniV2 liquidity, it is good to have some burning functionality jut so the owner can regulate the total supply in cases of emergency.

Validation steps

1. User gets the majority of tokens and performs malicious actions with them
2. Since there is no burning functionality he can perform malicious actions infinitely

Comments history

CurrenC team

This property is intentional and fundamental to the contract design.

The contract does not include burning functionality, as it would introduce administrative control over the supply.

The total supply is fixed at deployment and cannot be altered thereafter. No functions exist to modify supply, pause transfers, or intervene in user balances.

Introducing burn functionality would require privileged access or additional logic, increasing both attack surface and reliance on discretionary actions. Such mechanisms can create new vectors for misuse, error, or unintended side effects.

This constraint-based approach reduces complexity and eliminates dependency on “emergency” controls, which can themselves introduce centralization and security risks.

Accordingly, the absence of burning functionality is not a vulnerability, but a deliberate design decision. It ensures that no privileged entity can alter supply or intervene in response to user behavior, eliminating discretionary control, reducing attack surface, and minimizing trust assumptions.

ABOUT PROJECT

CurrenC is an independent research initiative focused on the design and publication of decentralized systems with minimal trust assumptions.

DISCLAIMER

HackenProof Disclaimer

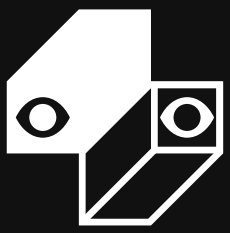
The application provided for assessment has been analyzed in accordance with generally accepted industry practices applicable at the time this report was prepared. The assessment focused on identifying cybersecurity vulnerabilities and security issues in the application's source code, deployment, configuration, and functionality, based solely on the materials, access, and scope made available for the engagement.

This report does not provide any representation or warranty that all vulnerabilities, weaknesses, or misconfigurations have been identified, nor does it guarantee the absolute security, reliability, or bug-free status of the application. The conclusions herein apply only to the specific version and scope reviewed and may no longer remain valid after any modification, update, or environmental change. This report should not be regarded as a definitive or exhaustive statement of the application's security posture.

Technical Disclaimer

Applications, whether decentralized applications (DApps) or other software systems, are deployed and operated within technical environments that may include blockchains, cloud services, operating systems, programming languages, frameworks, libraries, APIs, third-party services, and infrastructure components. Any of these elements may contain inherent weaknesses, undisclosed vulnerabilities, or configuration issues that can result in security incidents, operational failures, or unauthorized access.

The assessment reflects the condition of the application only at the time of testing and within the agreed scope, and the absence of findings in this report must not be interpreted as evidence that no vulnerabilities exist. Continuous security practices, including secure development, independent reviews, monitoring, patch management, and periodic reassessments, remain necessary to maintain an appropriate level of security.



HackenProof

Expert crowdsourced security provider

AUDIT

Contact us:

Linkedin | X | Website | Mail

2026